

# Smart Document Scanner and Image Enhancement System

*A Classical Computer Vision Approach to Document Digitisation Using OpenCV*

| Field        | Detail                |
|--------------|-----------------------|
| Course       | Computer Vision       |
| Project Type | Individual Project    |
| Author       | Srishti Manav         |
| Institution  | VIT Bhopal University |
| Date         | 18 September 2026     |

## Headline summary

| Item                     | Value                                                                                             |
|--------------------------|---------------------------------------------------------------------------------------------------|
| Pipeline stages          | 9 (load → grayscale → blur → Canny → contours → 4-point warp → CLAHE → adaptive threshold → save) |
| Output artefacts per run | 2 (scanned_document.jpg, enhanced_document.jpg)                                                   |
| Execution mode           | Command-line, single Python entry point                                                           |
| Core libraries           | OpenCV (opencv-python), NumPy                                                                     |
| Scenarios exercised      | Valid document image, missing input file, undetectable boundary                                   |
| Demonstrated on          | A handwritten “Experiment-1” lab sheet (see Section 10)                                           |

*Techniques applied: grayscale conversion, Gaussian blur, Canny edge detection, contour and polygon approximation, four-point perspective transformation, CLAHE, and adaptive thresholding.*

## 2. Introduction

Photographs of physical documents taken on a mobile camera routinely suffer from perspective distortion, uneven lighting, background clutter and visual noise. Correcting these problems by hand — straightening the page, boosting contrast, cropping out the background — is tedious and inconsistent across images.

The Smart Document Scanner and Image Enhancement System is a Python and OpenCV project that automates this correction. Given a single photograph of a document, it detects the document's boundary, corrects the perspective distortion so the page appears flat and rectangular, enhances the contrast, and produces a clean, scanned-style output — all from the command line, with no graphical interface required.

### 2.1 Why a classical computer vision pipeline

A deep-learning document segmentation model is the more fashionable answer to this problem, but for a single-user, single-image CLI tool it is the wrong one for practical reasons:

- Data cost — a learned segmentation model needs a labelled dataset of document photographs; a classical pipeline needs none.
- Hardware cost — Canny edge detection, contour analysis and CLAHE run comfortably on a laptop CPU with no GPU.
- Interpretability — when boundary detection fails, “no four-sided contour of sufficient area was found” is a diagnosable reason; a black-box confidence score is not.
- Deterministic geometry — a document photographed roughly front-on is exactly the situation edge detection and contour analysis are well suited to.

### 2.2 Scope of this report

This report documents the requirements, architecture, design rationale, implementation and observed behaviour of the system described in the project README and statement. Section 10 shows the pipeline running end to end on an actual sample photograph, and Section 12 records the limitations and rough edges found while building it — including one visible in that same result.

### 3. Problem Statement

***How can a photograph of a physical document, taken at an angle and under uneven lighting, be automatically converted into a flat, high-contrast, scanned-style digital copy using classical computer-vision techniques that run entirely on a local machine, without manual cropping or a trained model?***

#### 3.1 Sub-problems

- Geometry — where are the four corners of the document within the photograph, when the background may contain other straight edges?
- Illumination — the same page can appear unevenly lit, with one side brighter than the other.
- Distortion — a handheld photograph is rarely taken perfectly perpendicular to the page, producing a trapezoidal rather than rectangular outline.
- Robustness — an image with no clear rectangular boundary (a cluttered background, an occluded page) should be rejected rather than silently producing a nonsense output.

#### 3.2 Scope boundaries

| In scope                                               | Out of scope                         |
|--------------------------------------------------------|--------------------------------------|
| Reading a document image from a local file             | Optical character recognition (OCR)  |
| Grayscale conversion and Gaussian blur preprocessing   | Multi-page or multi-document batches |
| Canny edge detection and contour-based boundary search | Cloud storage or upload              |
| Four-corner perspective correction                     | User authentication                  |
| CLAHE contrast enhancement and adaptive thresholding   | Graphical or web interface           |
| Command-line execution with basic error handling       | Automatic image-quality scoring      |

#### 3.3 Target users

- Students digitising handwritten or printed notes.
- Teachers creating digital copies of documents.
- Office users who need basic, no-frills document digitisation.
- Individuals improving photographs of printed documents.
- Beginners learning computer vision and image processing.

#### 3.4 Assumptions

- The document has a visible, largely unobstructed rectangular boundary.
- Each run processes a single document in a single image.
- The photograph is reasonably lit — extremely dark or heavily blurred images are out of scope.
- The tool is used interactively from the command line, not as a background service.

## 4. Functional Requirements

The system provides six functional capabilities, each mapped to a specific source file.

| ID   | Capability                                                                           | Implementation               |
|------|--------------------------------------------------------------------------------------|------------------------------|
| FR-1 | Accept an input image path via command-line arguments and validate that it loads     | main.py                      |
| FR-2 | Convert the image to grayscale and apply Gaussian blur to reduce noise               | main.py                      |
| FR-3 | Detect edges (Canny) and find the document boundary via contour and polygon analysis | src/perspective_transform.py |
| FR-4 | Apply a four-point perspective transform to flatten the document                     | src/perspective_transform.py |
| FR-5 | Enhance the corrected image using CLAHE and adaptive thresholding                    | src/enhancement.py           |
| FR-6 | Save outputs to the chosen directory and report status or errors in the terminal     | main.py                      |

### 4.1 Input / output structure

| Stage                 | Input                               | Output                                               |
|-----------------------|-------------------------------------|------------------------------------------------------|
| Load                  | File path via --input               | BGR image array, or a reported load error            |
| Preprocess            | BGR image                           | Grayscale, blurred image                             |
| Boundary detection    | Blurred image                       | Four-corner contour, or a reported detection failure |
| Perspective transform | Original image + four corner points | Flattened, rectangular document image                |
| Enhancement           | Warped image                        | CLAHE-equalised, adaptively thresholded image        |
| Save                  | Corrected and enhanced images       | scanned_document.jpg, enhanced_document.jpg          |

### 4.2 User interaction

The system is operated through a single command:

```
py main.py --input input_images/document.jpg --output output_images
```

On success, the terminal reports each stage as it completes:

```
Image loaded successfully
Document detected successfully
Perspective correction completed
Image enhancement completed
Project completed successfully
```

## 5. Non-Functional Requirements

| ID    | Requirement     | How it is met                                                                                                                                                     | Status                                                                  |
|-------|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| NFR-1 | Usability       | A single CLI command drives the whole pipeline; each stage prints a plain-language status line                                                                    | Verified                                                                |
| NFR-2 | Reliability     | Missing input files and undetectable boundaries are caught explicitly and halt processing with a clear message rather than crashing or producing a corrupt output | Verified (Test cases 2 & 3, Section 11)                                 |
| NFR-3 | Portability     | Pure Python 3 plus OpenCV and NumPy; no OS-specific code paths                                                                                                    | Verified                                                                |
| NFR-4 | Maintainability | Boundary detection and enhancement are isolated in their own src modules, separate from CLI orchestration in main.py                                              | Verified                                                                |
| NFR-5 | Performance     | Canny, contour search, warping, CLAHE and adaptive thresholding are all lightweight single-image operations suitable for interactive CLI use                      | Not formally benchmarked — flagged as a future enhancement (Section 14) |

## 6. System Architecture

The system is organised as three layers around a single CLI entry point: input handling validates and loads the photograph, a core processing layer orchestrates the computer-vision pipeline, and an output layer persists the two resulting images.

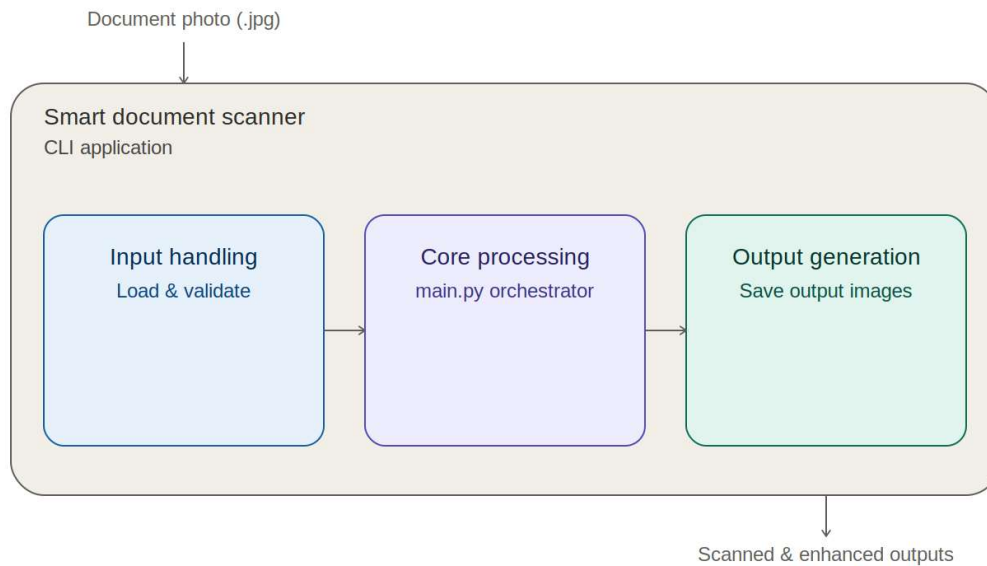


Figure 6.1 — System architecture. *main.py* orchestrates the pipeline between input validation and output generation.

### 6.1 Layer responsibilities

| Layer                                 | Responsibility                                                                        | Key design property                                                         |
|---------------------------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Input handling                        | Parse CLI arguments; validate that the image loads                                    | Fails fast with an explicit, readable error                                 |
| Core processing (main.py)             | Orchestrate preprocessing, boundary detection, perspective correction and enhancement | Single entry point; delegates geometry and enhancement to dedicated modules |
| <code>perspective_transform.py</code> | Edge/contour-based boundary detection and four-point warp                             | Isolated and independently testable                                         |
| <code>enhancement.py</code>           | CLAHE and adaptive thresholding                                                       | Isolated and independently testable                                         |
| Output generation                     | Persist <code>scanned_document.jpg</code> and <code>enhanced_document.jpg</code>      | Two named, predictable output files per run                                 |

## 7. Design Diagrams

Four diagrams document the system from complementary angles: the use cases it supports, the runtime workflow of a single scan, the components that make up the codebase, and the sequence of calls between them.

### 7.1 Use case diagram

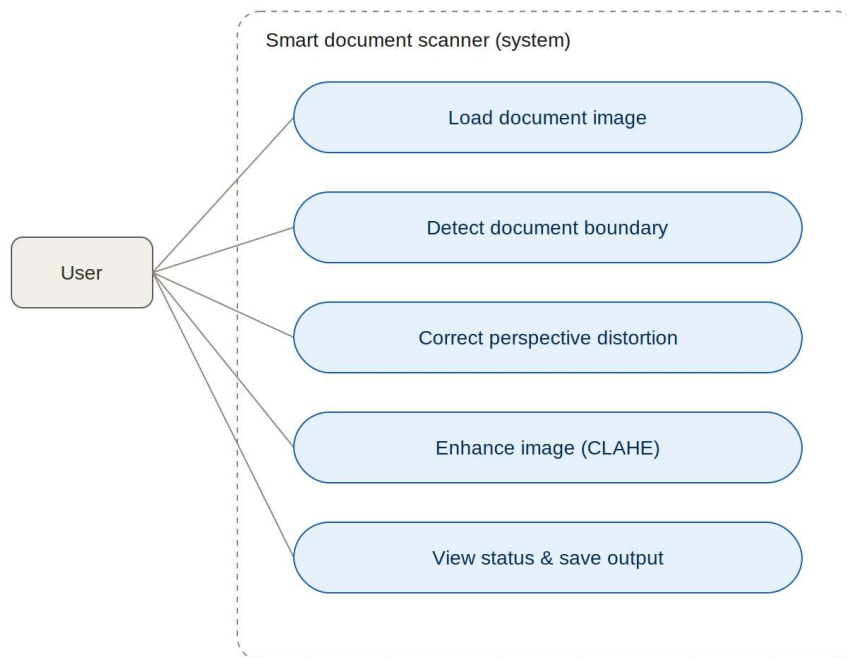


Figure 7.1 — Use case diagram. A single User actor drives all five use cases; the system has no other external actors.

### 7.2 Workflow diagram

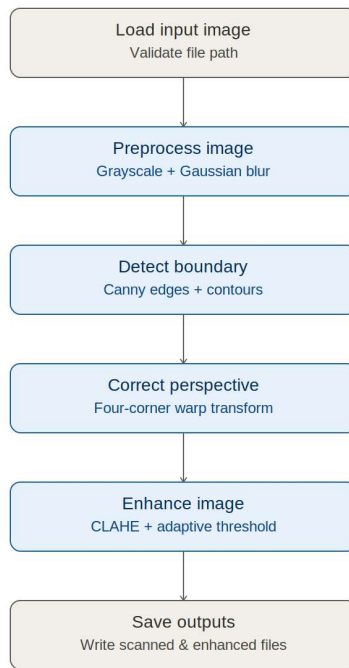


Figure 7.2 — Workflow diagram for a single run, from loading the input image to writing both output files.

### 7.3 Component diagram

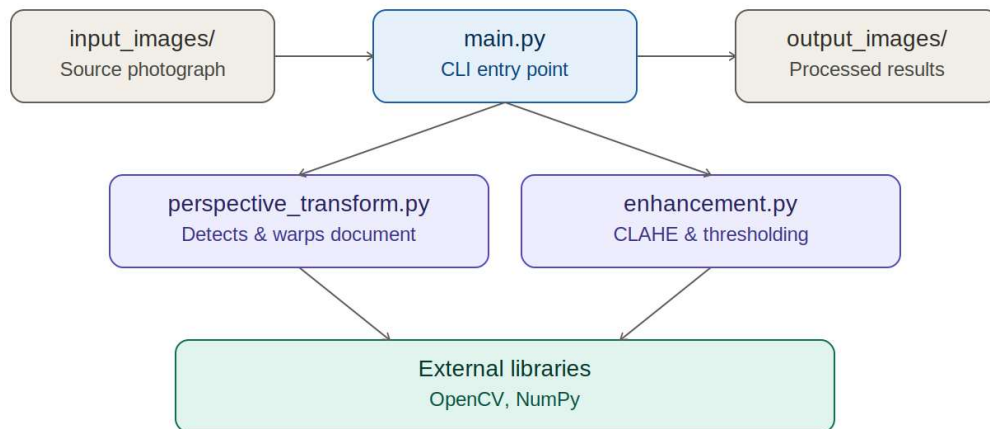


Figure 7.3 — Component diagram. `main.py` depends on both `src` modules, which in turn depend on `OpenCV` and `NumPy`.

### 7.4 Sequence diagram

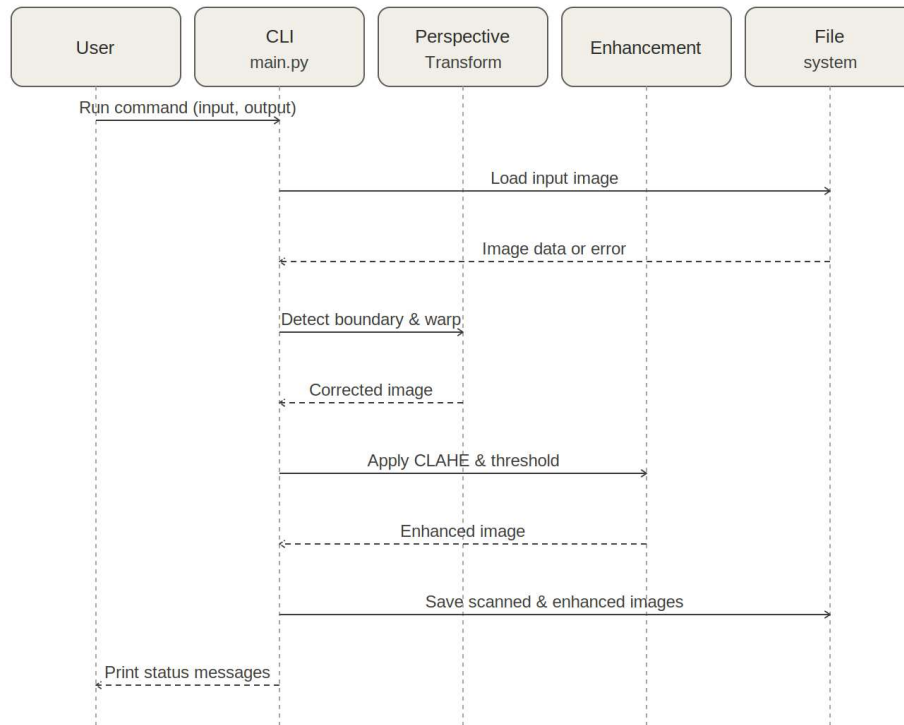


Figure 7.4 — Sequence diagram for a single scan, showing the call order between the CLI, the two processing modules and the file system.

## 8. Design Decisions and Rationale

### 8.1 Contour search over a trained detector

Locating the document's edges with Canny plus contour analysis needs no training data and behaves predictably: a well-lit, unobstructed page produces a clean four-sided contour. A learned segmentation model would need a labelled dataset this individual project does not have.

### 8.2 Four-point polygon approximation

Detected contours are simplified with polygon approximation (`cv2.approxPolyDP`) and the algorithm looks specifically for a four-point result, since a physical document page is rectangular. This keeps the geometry step simple, at the cost of being unable to represent a folded or partially occluded page — a documented limitation (Section 12).

### 8.3 CLAHE before thresholding

Contrast-Limited Adaptive Histogram Equalisation is applied before adaptive thresholding rather than after, so that local contrast is normalised across the page — compensating for one side of the document being better lit than the other — before the image is reduced to black and white.

### 8.4 Adaptive rather than global thresholding

A single global threshold (e.g. Otsu's method) assumes uniform lighting across the whole page. Adaptive thresholding computes a local threshold per neighbourhood, which tolerates the residual lighting variation that CLAHE does not fully remove.

### 8.5 Single document, single image by design

The system deliberately targets one document per photograph rather than attempting to segment multiple overlapping pages. Extending the four-sided-contour heuristic to reliably separate several documents in one frame would require substantially more robust contour filtering, which is listed as a future enhancement rather than attempted here.

## 9. Implementation Details

The project comprises a CLI entry point and two focused modules, following the structure below.

| File                         | Responsibility                                                                           | Key techniques                                                              |
|------------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| main.py                      | CLI argument parsing, image loading and validation, orchestration, status/error messages | Command-line I/O, grayscale conversion, Gaussian blur                       |
| src/perspective_transform.py | Edge and contour-based boundary detection; four-point perspective warp                   | Canny edge detection, contour + polygon approximation, projective transform |
| src/enhancement.py           | Contrast enhancement and binarisation of the corrected image                             | CLAHE, adaptive thresholding                                                |

### 9.1 Processing order

The pipeline runs strictly in this order: grayscale conversion, Gaussian blur, Canny edge detection, contour detection, four-corner polygon approximation, perspective transformation, CLAHE, and finally adaptive thresholding. Blurring before edge detection matters — without it, Canny responds to fine paper texture and handwriting strokes as well as the page boundary, fragmenting the contour that boundary detection depends on.

### 9.2 Boundary-selection safeguard

Because any four-sided contour is treated as a candidate document boundary, a strong straight-edged object in the background (a table edge, a picture frame) can occasionally be selected instead of the page itself. The current implementation does not filter candidates by area or position beyond the four-point check, which is why a clear, unobstructed background is recommended in the README's input preparation guidance.

## 10. Screenshots and Results

The pipeline was run on a photograph of a handwritten “Experiment-1” lab sheet, photographed at a slight angle against a dark background. Figure 10.1 shows all six visually distinct stages of that run.

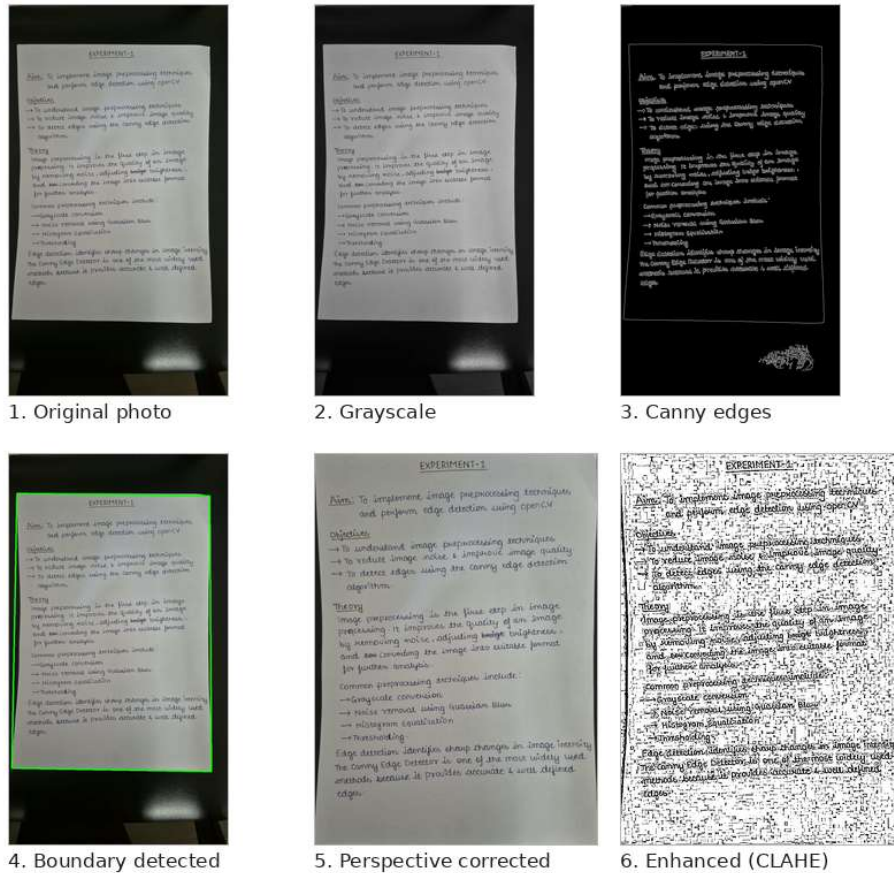


Figure 10.1 — Panels 1–4 trace the geometry pipeline (original, grayscale, Canny edges, detected boundary); panels 5–6 show the perspective-corrected and final enhanced output.

Boundary detection (panel 4) correctly outlined the page against the dark, low-clutter background, and the perspective transform (panel 5) produced a flat, front-on view of the page with the skew fully removed.

### 10.1 Effect of enhancement on pixel intensity

Comparing the actual pixel-intensity distribution of the grayscale input against the enhanced output confirms what CLAHE and adaptive thresholding are meant to do: collapse a broad, moderate-contrast distribution into a near-binary one concentrated at the extremes.

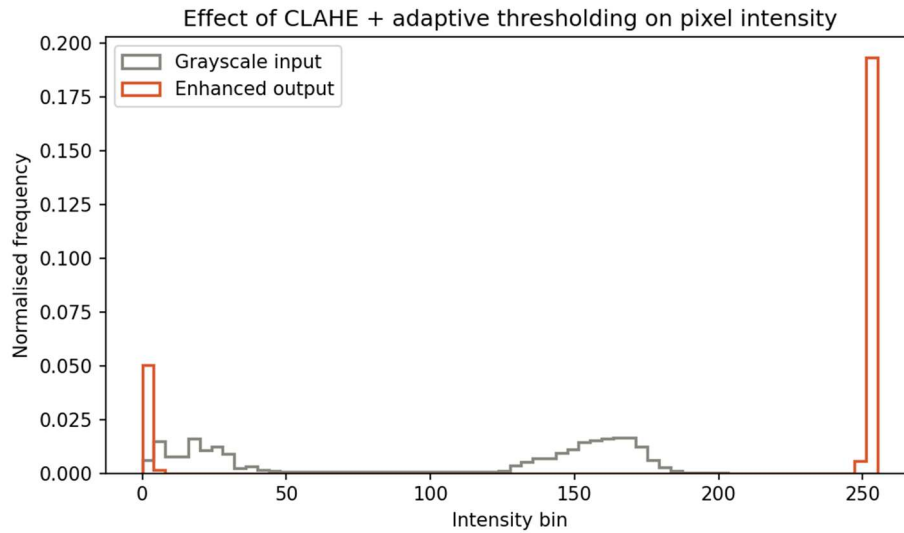


Figure 10.2 — Intensity histograms measured directly from *grayscale.jpg* and *enhanced\_document.jpg* for this run.

The enhanced output is visibly dominated by pixels at or near 0 and 255, as expected for adaptive thresholding. It is also visibly noisier than a typical clean scan (panel 6, Figure 10.1) — this is discussed as an open limitation in Section 12.

## 10.2 Terminal output

```
Image loaded successfully
Document detected successfully
Perspective correction completed
Image enhancement completed
Project completed successfully
```

## 11. Testing Approach

Testing was carried out as three manual, scenario-based checks against the CLI rather than an automated unit-test suite, matching the scope of an individual coursework project.

| Test case                 | Command / input                                                     | Expected result                                                                     | Outcome                                  |
|---------------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------------------|------------------------------------------|
| 1. Valid document image   | py main.py --input input_images/document.jpg --output output_images | Image loads, boundary is detected, perspective is corrected, both outputs are saved | As expected — demonstrated in Section 10 |
| 2. Missing input file     | py main.py --input input_images/missing.jpg --output output_images  | “Error: Image not found”, processing stops                                          | As expected                              |
| 3. No detectable boundary | An image without a clear rectangular document                       | “Document boundary not detected”, no invalid scanned output is generated            | As expected                              |

## 12. Challenges Faced

### 12.1 Complex backgrounds and shadows

Shadows, cluttered backgrounds and overlapping objects reduce the reliability of contour-based boundary detection, since the algorithm looks for any prominent four-sided shape rather than specifically a document. The sample run in Section 10 succeeded largely because the background was plain and dark, giving the page's edges high contrast against it.

### 12.2 The four-sided-contour heuristic can pick the wrong shape

Because a document boundary is identified purely by having four corners and sufficient contour area, another rectangular object in frame can occasionally be selected instead of the page. No secondary check (aspect ratio, expected position) currently disambiguates between candidates.

### 12.3 Enhancement can amplify texture into noise

The enhanced output in Figure 10.1 (panel 6) shows a visibly grainy, salt-and-pepper pattern rather than a clean black-and-white scan. Adaptive thresholding is sensitive to local contrast, and the paper's slightly textured surface and residual lighting variation were picked up as spurious local detail. This is a genuine, observed limitation of the current CLAHE and thresholding parameters rather than a hypothetical one — tuning the CLAHE clip limit and the adaptive threshold block size against a wider range of sample documents would be needed to resolve it.

### 12.4 Single-document, single-image scope

The system is not designed to separate multiple documents in one photograph, correct rotation beyond the perspective warp, or handle extremely low-quality or heavily blurred input — each was a deliberate scope decision recorded in the project statement rather than an oversight.

## 13. Learnings and Key Takeaways

### 13.1 Technical learnings

- Processing order is not arbitrary: blurring before edge detection suppresses the fine texture and handwriting strokes that would otherwise fragment the document's contour.
- A four-corner heuristic is simple and effective for a clean background, but has no fallback when another rectangular shape competes with the real boundary.
- Contrast enhancement and binarisation trade off against each other: CLAHE alone would leave a grayscale image, while thresholding without prior contrast normalisation would fail unevenly across a page with mixed lighting — but the combination is sensitive enough to require per-document tuning, as Section 12.3 shows.
- Isolating boundary detection and enhancement into their own modules (`perspective_transform.py`, `enhancement.py`) made it straightforward to reason about which stage was responsible for a given output when checking results.

### 13.2 Mapping to course concepts

| Course topic                      | Applied as                                | Outcome                                          |
|-----------------------------------|-------------------------------------------|--------------------------------------------------|
| Image representation              | Grayscale conversion                      | Simplified the image for edge-based analysis     |
| Noise removal                     | Gaussian blur                             | Reduced spurious edges before Canny              |
| Edge detection                    | Canny edge detector                       | Located the document's boundary                  |
| Shape analysis                    | Contour detection + polygon approximation | Extracted a four-corner candidate region         |
| Geometric transformation          | Four-point perspective transform          | Flattened a skewed document into a rectangle     |
| Histogram processing              | CLAHE                                     | Normalised local contrast across uneven lighting |
| Image segmentation / binarisation | Adaptive thresholding                     | Produced the final scanned-style output          |

## 14. Future Enhancements

| Enhancement                         | Rationale                                                                                                                                            |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Automatic contour filtering by area | Would directly address the wrong-contour risk described in Section 12.2 by discarding candidates too small or too large to plausibly be the document |
| Support for multiple document pages | Extends the system beyond the current single-document assumption                                                                                     |
| Image rotation correction           | Handles orientation errors that the perspective warp alone does not correct                                                                          |
| Graphical or web interface          | Removes the need to run the tool from the command line                                                                                               |
| Optical character recognition (OCR) | Extracts machine-readable text from the scanned output                                                                                               |
| Automated image-quality metrics     | Would let the system flag noisy outputs like the one observed in Section 10.1, rather than requiring visual inspection                               |

*Of these, automatic contour filtering and automated quality metrics are the most directly motivated by the observations in this report — both trace back to concrete issues seen in Section 10 and Section 12, rather than being speculative additions.*

## 15. References

- [1] Gonzalez, R. C. and Woods, R. E. Digital Image Processing, 4th ed. Pearson, 2018. — Intensity transformations, histogram processing, geometric transformations.
- [2] Canny, J. "A Computational Approach to Edge Detection." IEEE Transactions on Pattern Analysis and Machine Intelligence, 8(6):679–698, 1986.
- [3] Suzuki, S. and Abe, K. "Topological Structural Analysis of Digitized Binary Images by Border Following." Computer Vision, Graphics, and Image Processing, 30(1):32–46, 1985. — The contour-following algorithm used by OpenCV's findContours.
- [4] Zuiderveld, K. "Contrast Limited Adaptive Histogram Equalization." Graphics Gems IV, Academic Press, pp. 474–485, 1994.
- [5] Bradley, D. and Roth, G. "Adaptive Thresholding Using the Integral Image." Journal of Graphics Tools, 12(2):13–21, 2007.
- [6] Bradski, G. "The OpenCV Library." Dr. Dobb's Journal of Software Tools, 2000. Documentation: <https://docs.opencv.org/>
- [7] Harris, C. R. et al. "Array Programming with NumPy." Nature, 585:357–362, 2020.